Concepts, activities and issues of policy-based communications management

M D J Cox and R G Davison

Policy-based management is an approach that has been considered for some years within the distributed systems management research community. The concepts have recently been adopted by standards bodies including the International Telecommunications Union (ITU), Internet Engineering Task Force (IETF), Desktop Management Taskforce (DMTF) and Object Management Group (OMG) and are starting to appear in marketing literature for Internet protocol (IP) network management products. The meaning of the term is not well-defined nor consistently used, but common aspects of policy include that it changes a system's behaviour, that it is defined after system deployment, and that it does not require a full software development life cycle. Technical problems remain which need to be solved before the full ambitions of policy-based management can be achieved. Initial products are likely to address only a sub-set of the full potential and to exist in niche domains.

1. Introduction

Today's network operator is under increasing pressure to change networks and systems in response to changing customer demands, services and business goals. This requires more flexible systems that can be changed more easily. Network technology has already become flexible and can be configured and easily re-configured to support different mixes of services with different bandwidth and quality of service (QoS) requirements. To exploit this range of behaviours, communications management systems need to be as flexible.

The ideas of policy and policy-based management [1] have been proposed as a means to:

- provide more flexible management systems that can respond to rapid change in requirements by run-time reconfiguration rather than re-engineering,
- hide complexity by bridging the gap between business objectives and network level configuration.

The fundamental premise of policy-based management approaches is that there is some system behaviour which will need to be changed during a system's lifetime and that behaviour can be separated out from the system. The behaviour can then be changed by the system user without changing the system implementation. The behaviour is expressed as a set of 'policies' which are defined at a level of abstraction that hides system complexity.

Achieving policy-based management is not trivial and requires a number of difficult issues to be addressed including provision of:

- applications with run-time flexibility — building management systems that can be reconfigured easily to provide different behaviour,
- an abstract view for human managers — providing an abstraction of what is being managed that hides the complexity,
- a management interface to automation — providing an interface through which the automatic behaviour of applications can be directed by human decision making,
- distribution of configuration — the broadcast of configuration information to distributed entities,
- consistent re-configuration of distributed entities — consistently setting up independent players so that they meet some objective,
- customising bought-in components to meet the operator's goals — the initial set-up of vendor-supplied components so that they meet an operator's goals.

Policy-based management has been a topic primarily of academic research since the early 1990s. It has been given a recent boost through its adoption by standardisation bodies such as ITU, OMG, IETF and the DMTF, and through the appearance of products particularly in the Internet protocol

(IP) network management domain. However, an examination of the various activities does not give a clear understanding of what a policy is and what policy-based management provides that is new. The lack of clear definitions leaves the opportunity for marketing hype and confusion.

In this paper an attempt is made to provide an understanding of what policy-based management is and why it would be useful, to give an overview of initiatives, and to identify the key issues. Firstly section 2 of the paper will clarify what a policy is. Section 3 will then discuss policy-based management in more detail. Section 4 provides a summary of some of the research and standardisation that has taken place (the appendix provides some more detail). Section 5 provides an indication of what has been achieved and what the main issues are, while section 6 draws some conclusions about the immediate potential for policy-based management.

2. What is policy?

Policy-based management is a research topic and as such does not have clear and agreed definitions. Consider the following selection of definitions of the term 'policy' extracted from the literature:

- the plans of an organisation to meet its goals [2],
- objects which define a relationship between managers and managed objects [1],
- policies are derived from the goals of management and define the desired behaviour of distributed heterogeneous systems and networks [3],
- information which influences the behaviour of managers and managed objects [4],
- policy is a point somewhere between ... management goals and plans significant only in that difficult decisions have been made leaving only easy decisions [5],
- an identifiable specification which can be evaluated with respect to managed objects [6].

This section contains an attempt to be more precise about what a policy is — firstly a definition:

> 'A statement about a system's behaviour that will be translated and then incorporated into the system to change its behaviour at run time.'

This definition, although correct, is not sufficient. It does not say who sets policies, why, what system is affected, and how. The remainder of this section will refine this statement further by considering these factors.

A common, though not always explicit, characteristic of policy-based approaches is that the policies are not written by the system developer. Typically policies are written by the system user who, in the case of communications management, would be a network manager.

The system user, with the aid of a policy support system would set a policy and send it to the management system via a policy interface. The management system then interprets and enforces those policies which affect the behaviour of the management and managed systems. In the particular case of network management, the target managed system is the network and the subject management system is the management and control function. Figure 1 illustrates the different systems involved and their interfaces.

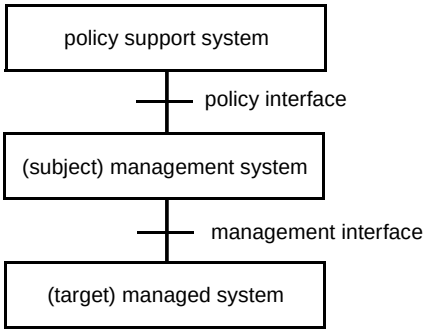

Fig 1 Systems and interfaces.

The intention of policies is to be the means by which the behaviour of the management system can be altered during its lifetime. The need to do this arises whenever a decision about alternative behaviours of the system depends on information not available to the system itself. This implies a design decision taken by the system developer to separate behaviour that is fixed from behaviour that needs to be dynamically disabled and enabled by policy.

However, the intention of policies is not to represent one-off management commands. Management commands will result in an immediate action being taken by a system but once the action is taken the command has been completed and ceases to exist. In contrast, policies are incorporated into a system and continue to affect that system until revoked or replaced, i.e. they are persistent.

A key intention behind policies is that a system does not require re-testing after a policy has been incorporated into it. If this cannot be achieved, policies have little advantage over conventional software enhancement. The policy system must be designed so that a new policy will not make the system crash. This criteria does not prevent bad policies from being created. It is possible to produce one or a set of policies that cannot be achieved, for example to maintain network utilisation at 99% while rejecting 20% of connections. The system must be designed so that if this happens the conflict is detected, the system goes to a well-

behaved error state and a conflict resolution function is instigated to correct the policies.

A confused area in policy literature is whether policies can add new behaviour to a system or whether they can just select or configure a set of pre-existing behaviours. A lot of the confusion comes from the interpretation of words like behaviour. In our view, the key to understanding the issue is to consider it in the light of who produces what in the system. The system manufacturer provides a mechanism through which policies can influence the system operation. The nature and power of this interface will vary but essentially there will be some fixed functionality within the application which is available for policy-driven use. A policy cannot extend the available fixed functionality, but, depending upon the interface provided, may be able to create new control paths through it. This new path could be viewed as new behaviour from outside the system, but, from the manufacturer's viewpoint, has not added any new primitive behaviour. This distinction is closely linked to the desire that adding new policies should not require system testing whereas adding new fixed functionality would.

3. Concepts and characteristics of policy-based management

The previous section described what a policy is. This section will describe some of the concepts and characteristics of policy-based management, in particular:

- how a policy is processed by discussion of the policy life cycle,
- the range of policy representation.

3.1 Policy life cycle

Policies evolve through a policy life cycle [7]. Figure 2 illustrates a typical a policy life cycle and the following text explains the main phases.

The first stage in the life cycle is specification of policy. Here the system user writes a policy in some language and it becomes represented in some form within the policy support system. Following Moffett and Sloman [2], a policy may typically have the following attributes:

- subjects — the set (domain) of managers that must apply the policy,
- modality — whether policy is an authorisation policy permitting or forbidding certain action or an obligation policy forcing or deterring a certain action,
- trigger — an optional event which will trigger application of the policy,

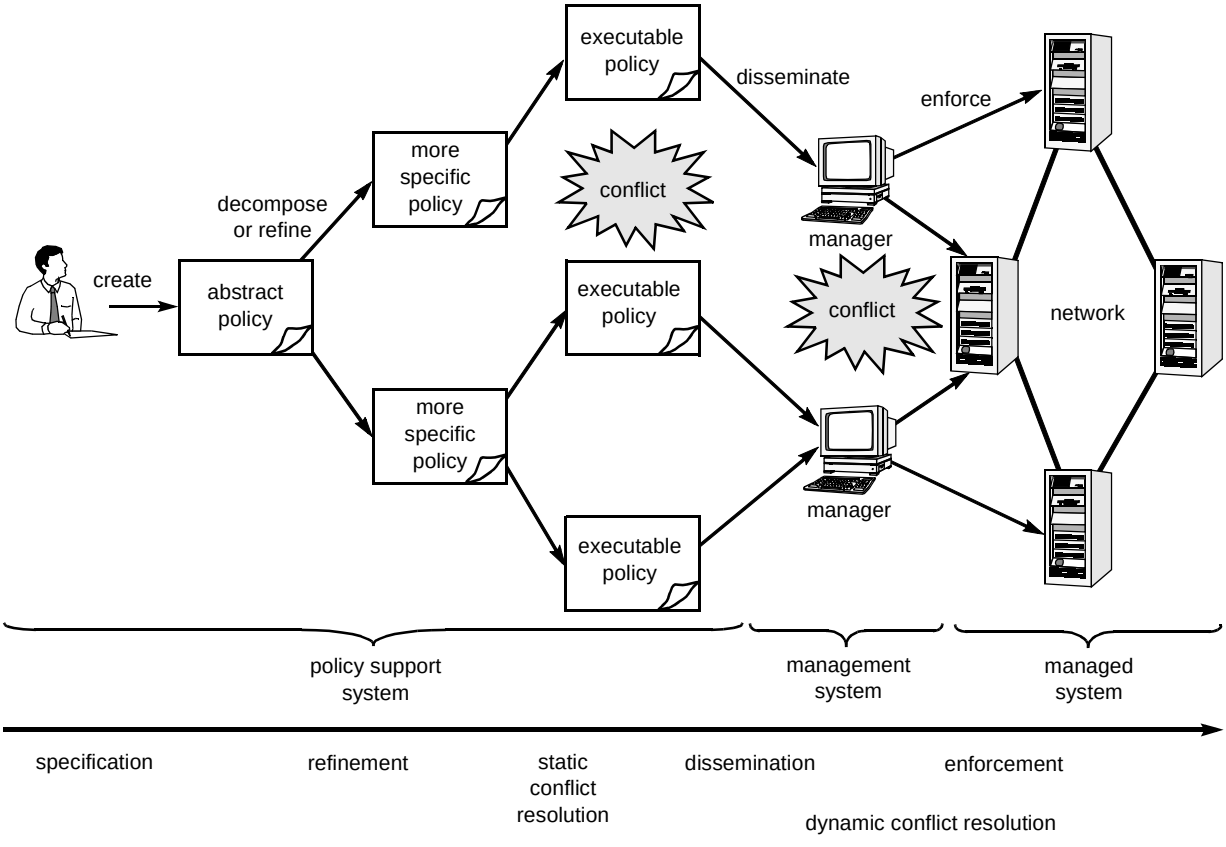

Fig 2 Policy life cycle.

- action — the management action or actions affected by the policy,
- targets — the set (domain) of managed objects at which the policy is directed,
- constraints — predicates that must be satisfied before the policy can be applied.

Initially high-level abstract policies are specified but are made gradually more detailed in a semi-automatic process of policy refinement until they are in a form that can be interpreted and enforced by the management system. This is a difficult process involving decomposition of larger goals, defining procedures to meet goals, expanding domains of subjects and targets to instances, and generally turning abstract statements into device-specific executable instructions. This leads to a data structure of successively more detailed policies known as a policy hierarchy.

During this process some policy analysis occurs, in particular policy conflict. Policy conflict can occur when two policies cannot be simultaneously met. Detection of policy conflict may be conducted off-line (static) by examining the content of policy statements. Often only the potential for conflict can be detected off-line because the members of subject and target domains are not known until run time. The potential for conflict may be tolerated or may require a resolution strategy to choose the most appropriate policy. Conflict resolution can take place off-line by editing policies, prioritising policies, or providing meta-policies.

The next stage in the policy life cycle is that of policy dissemination, i.e. the means by which the policies are distributed to managers or by which the managers look up centrally held policies. Once policy information is distributed to the management system some mechanism is needed in order to undertake policy enforcement. This requires that the policy information is translated to an executable form, is checked at the correct point in the management system function, and, depending on the result, then selects certain actions over others. During the course of enforcement, (dynamic) policy conflict may be detected through a manager (or managers) discovering that they have two contradictory policies to enforce. Such conflict may simply be reported to the policy support system or may be resolved by applying previously defined prioritisation or meta-policies that choose which policy to apply.

Later stages of the life cycle may also include stages for disabling, withdrawing and deleting policies.

3.2 Policy representation

Each approach to policy-based management includes a language of some form which is used to define the policies. The representation of policies can vary according to:

- the level of abstraction used,
- the type of statement that can be expressed and the representation approach used.

3.2.1 Level of abstraction

Policies may exist at different levels of abstraction, i.e. in the level of detail they provide. Moffet and Sloman [8] introduced the idea of a policy hierarchy to represent policies at different abstractions as they are refined through a policy system. Various authors attempt to give structure to this hierarchy as shown in Fig 3.

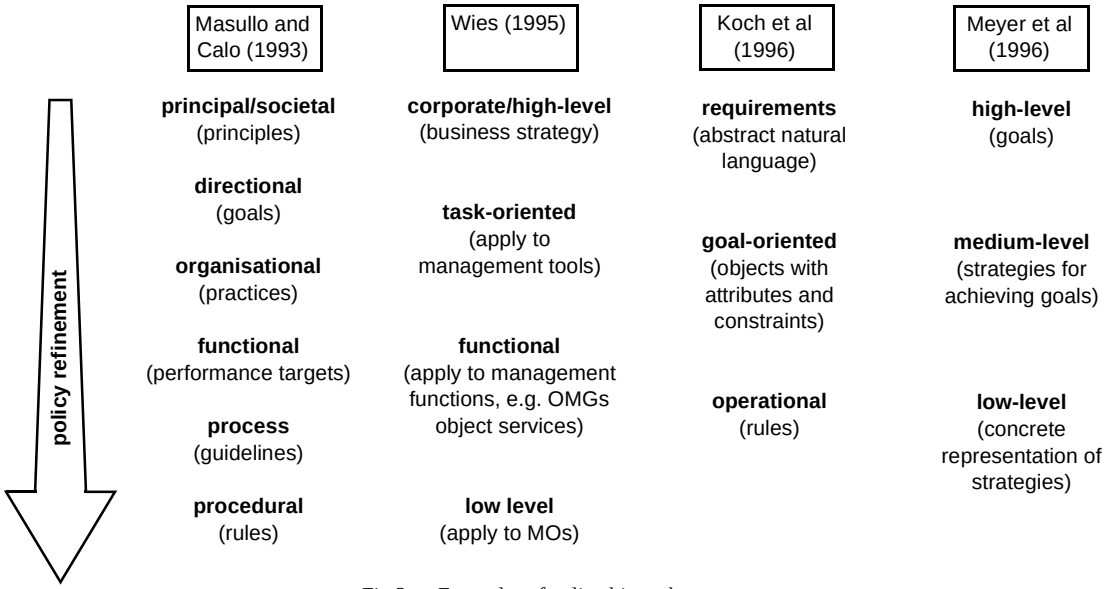

Fig 3 Examples of policy hierarchy structure.

For network management an example of a possible policy hierarchy (Fig 4) is described below:

- human-friendly — at this level, policy is expressed in a form that is easy to read and is independent of network equipment details, e.g. policies may refer to IP routers but would not be aware of vendor or model differences (policies at this level may apply network-wide),

- equipment-independent — at this level policy is independent of equipment details but is not easy for people to read,

- equipment-specific — at this level policies are specific to a particular piece of equipment (these policies cannot be network-wide),

- executable — at this level of abstraction policies have become executable code or instructions to pass to the target system.

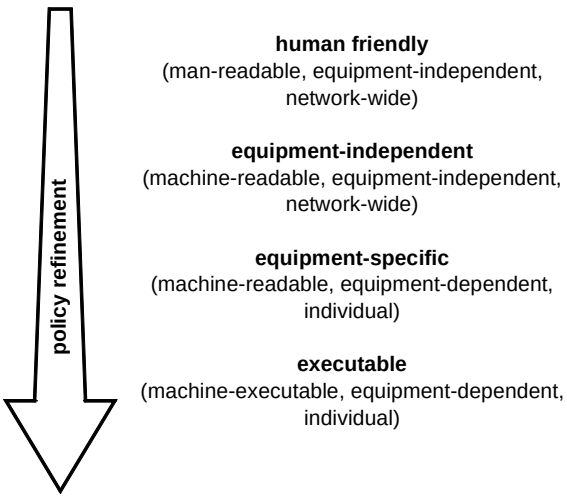

Fig 4 Possible policy hierarchy for network management.

The level of abstraction is an important factor in comparing and understanding approaches. The more abstract a permitted policy is, the greater the sophistication of the policy support system needed to map that policy into a target system.

3.2.2 Policy expression and representation

One particular sort of difference in abstraction is related to the expressiveness of this language. Based on ideas in Becker and Holden [5] the following informal characterisation of policy statements has been developed, each representing a different level of detail concerning management actions affected by policy.

- Constraints on states

  This category contains policies that express system states that must or must not be achieved. Such policies consist of logical statements comparing variables to values to specify state regions that must or must not be maintained. Examples could be: ‘Network utilisation < 0.7’ or ‘If it is a Sunday then keep utilisation below 0.8’

- Constraints on actions

  This category expresses limitations on what actions can or cannot be taken. This requires only that an existing action can be specified along with the conditions which determine its use or not. This sort of statement requires at least that the application check, before the action is done, that it is required or permitted. Examples could be: ‘User A is not permitted to access the Internet’, or ‘Access control must log accesses to the Internet’.

- Plans of action

  This category contains policies that allow sequences of possibly conditional primitive actions. Plans of action can be used as macros to define new actions in terms of the primitive actions. This type of statement requires both a policy language with control structures approaching that of a programming language, and an enforcement mechanism involving some kind of interpreter. However, the burden that it places on conflict detection and resolution would be significantly higher than with other types of policy. Examples could be: ‘If network utilisation exceeds 0.6, decrease acceptance thresholds and inform manager’, or ‘If asked to tear down link, stop new connections, wait for existing connections to complete and then set linkstate to disabled’.

The above categories represent a useful way of categorising different policies although there are considerable overlaps. More precise categorisations will emerge as the field matures.

Different groups have taken alternative approaches to representing policies. Many have used a rule-based approach, but approaches such as scripting languages or conventional programming languages such as Java have also been used. Some approaches use different representations depending on where the policy is in its life cycle. As the field matures, it is reasonable to expect some convergence in these approaches although it does seem unlikely that a universally accepted language would appear. It is hoped that there will be consistency within problem domains, e.g. a single language expressing network management policies may appear in time. Equally important as the language is the existence of common models or ontologies with agreed semantics for the languages to manipulate. For example, the existence of a standard set of

asynchronous transfer mode (ATM) management objects for policies to manipulate would be advantageous for achieving vendor-independent management, but would be as difficult to achieve as agreeing standard management information bases (MIBs).

4. Example — policy-based management of virtual path networks

In this section an example is depicted to try to make these ideas more concrete. Application areas where policy-based management comes into its own involve a great amount of choice about how some complex system, composed of many independent functions, can work. In such domains there is often the need to align the configuration of these independent functions so that they serve the business objectives. Examples of such areas in the telecommunications domain include configuration management of network elements, security access control to management or services, event filtering and routeing from network elements, and network resource allocation.

Consider, for example, the goals of maintaining performance of a multiple QoS broadband network such as an ATM network. This is a problem of resource allocation where the aim is to maximise the use of network bandwidth while needing to meet a number of different service level agreements which commit to performance guarantees, such as connection rejection rates, delay, loss, and jitter. This problem could be addressed by over-providing resource but in this example the resource will be deemed to be scarce and require careful management. This example will consider those policies affecting the operation of a bandwidth management function which is concerned with ensuring traffic demands are met with sufficient but not excessive bandwidth.

In ATM networks a logical overlay network of virtual paths (VPs) can be created between a source and a destination typically carrying a particular traffic type. Each reserves some bandwidth on every link on a given route between the source and destination. It can happen though that one VP is in demand and congested while others that share the same physical links are empty. The purpose of the bandwidth management function is to reallocate bandwidth from the empty VPs to the congested VPs. It may be necessary to regulate the operation of the bandwidth management with policies, e.g. to specify under what conditions bandwidth is to be allocated or deallocated (see Fig 5).

Typically we wish to express policies at a high level of abstraction for the convenience of the operator and (hopefully) automatically refine them into detailed machine interpretable statements. Figure 6 illustrates an example of how an abstract policy of ensuring network performance might be refined down to a specific policy. The bold text indicates an abstract part of the policy being refined into a detailed one. Here the very abstract performance management policy is refined into a specific policy for a bandwidth manager function to increase the bandwidth available to a QoS class (by a certain amount) when existing bandwidth becomes more than 70% utilised. This involves applying a number of types of refinement (based on Moffett and Sloman [8]), such as:

- goal refinement — decomposition of a larger goal into sub-goals necessary to achieve the goal,
- procedural refinement — identification of action(s) necessary to achieve a goal,
- delegation — identification of a particular group of managers responsible,
- target partitioning — identification of a particular group of target resources to be affected,
- domain resolution — identifying individual subjects and targets affected.

It is not hard to see that the scope for automatic support of some of these steps, in particular those higher up the policy hierarchy, is quite limited. Procedural and goal refinements can require human experience to achieve. Others though are more amenable to automatic support such as resolving domains.

With such flexibility comes the potential for conflict, i.e. setting two policies that cannot both be met. It is desirable to detect such conflicts and resolve them. With some types of conflict it is possible to detect them off-line by examination of the policies, but generally only the potential for conflict can be detected. This is because the subjects and targets affected are not evaluated until run time, and actual conflict occurrence depends on external factors, e.g. the number of connections that actually arrive. In many cases the semantics of the actions need to be understood in order to find conflict off-line, e.g. that increasing and decreasing a VP bandwidth are contradictory actions.

Manager must on Until > 70% do Increase VP by 10% on lowQoS VPs when off-peak

<Subject> <Modality> <Trigger> <Action> <Targets> <Constraints>

Fig 5 Example bandwidth management policy.

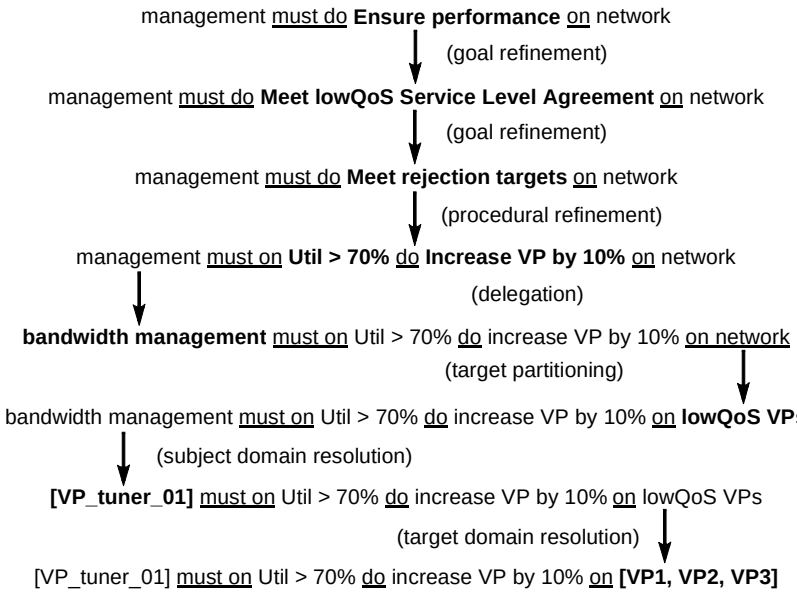

Fig 6 Policy refinement example.

For an example of conflict consider an obligation policy on the bandwidth manager to decrease VP bandwidth of all VPs and an authorisation policy forbidding the bandwidth manager to reduce the bandwidth of some VPs. Here for some VPs bandwidth management is not permitted to take an action that it has been obliged to perform:

```
Bandwidth management must on util < 0.5 do
decrease VP by 10% on all_VPs

Bandwidth management can't do decrease VP on
fixed_bandwidth_VPs
```

In this case the conflict could be resolved by adopting a strategy of domain nesting precedence whereby the authorisation rule takes priority because its domain of fixed_bandwidth_VPs is a sub-domain of all_VPs or the obligation policy could simply be edited so that it excludes these VPs.

What happens in policy dissemination is relatively straightforward in terms of purpose. It is simply the mechanism of broadcasting the policy to the managers who will interpret it from the policy repository. In practice, of course, there are many issues to address — for example, should the manager consult the policy service for a relevant policy when it needs to make a policy decision, or should policies be disseminated to a local repository when created or updated. So, for example, bandwidth management could receive an indication of VP utilisation and then consult a policy server for an indication of what action to take, if any, or it may refer to a local cache providing the same information.

In terms of interpretation and enforcement there are different ways in which a disseminated policy can be interpreted and enforced. These are only examples — there may be other options. The policy above may amount to setting the values of attributes in some data structure that will be checked by the bandwidth manager at the appropriate point in its function, for example:

```
LowQoS.LOW_UTIL_THRESHOLD = 0.5
LowQoS.DECREASE_BW_PROPORTION = 10
```

or it may require that a rule is created which will be applied by a rule-based interpreter. On each cycle the interpreter will check the conditions on the left-hand side of the rule and if true call the management action, for example:

```
IF   VP.utilisation < 0.5
THEN VP.decrease_VP_bandwidth(10%)
```

5. Summary of activities

Policy-based management has been a particular focus of the research community since the early 1990s. Some of the ideas have now filtered into standards bodies such as OSI, IETF, DMTF and OMG in a limited fashion. More recently policy-based products have been appearing for IP network management.

In the research work, there appears to be an emphasis on solving the abstract problem with much discussion of concepts but relatively little experience of practical application to particular domains. The difficult areas of conflict analysis and, in particular, policy refinement require more research. The scope of standards activities often excludes consideration of the difficult issues and often focuses on specific domains of application and specific types of policy. They concentrate on policy specification, interfaces, and dissemination mechanisms rather than

dwelling on good definitions of policies, method of enforcement, refinement and conflict resolution, and tools for the policy support system. Some vendor products are also starting to appear in niche domains. These products will be useful but purchasers should be wary of limitations not stressed in the marketing literature

What is clear over all the activities is that there is a great deal of diversity in approach to all aspects of policy-based management and no good clear definitions have emerged. The major differences are listed below.

- Policy definition

  Research efforts try to produce a generic definition which can be vague and confusing. Standards bodies side-step this difficult issues and move on to specify particular sorts of policy.

- Management scenario

  There are big differences between policy-based application management, policy-based network management and policy-based networking in terms of who sets the policy, how it is enforced and what managed resources are affected.

- Scope of the policy representation

  Research approaches are often concerned with the abstract problem and describe generic policy languages. More practical approaches such as those of the standards bodies tend to define the common attributes of policy but require specialisation to describe specific policies. Standards bodies have to define a domain ontology which limits policy to that domain.

- Power of representation

  Earlier we identified a possible categorisation of policy into goals, constraints on action, or plans of action — each with more expressive language. Different approaches in the literature to representing policy can be positioned as to whether they are more or less like these categories.

- Level of abstraction

  The range of level of abstraction catered for in the policy representation can vary. This is an important factor because to include more abstract descriptions implies that there are either intelligent automated or human managers to interpret the policy, and that there is refinement functionality to turn the abstract policies into more detailed policies. Given the lack of much substantial work on policy refinement, standards bodies tend to be at a particularly low level while academic efforts attempt to embrace a range of abstraction levels.

- Treatment of domains

  Most activities incorporate the concept of a domain, i.e. a set of objects grouped for the purposes of management. The implementation varies significantly though — for example, whether domains are purely a means of implementing a policy or they are a management service in their own right, whether they can be defined with predicates or have to be explicitly enumerated, whether they are allowed to overlap, and whether they have sub-domains.

- How policies are enforced

  There are different ways of interpreting and enforcing policies which permit different degrees of power to the policy setter. In some approaches policies can enable and disable particular behaviours at certain key points in management function. In other approaches the policies becomes rules that define the action of the manager because the fixed functionality in the manager is limited to a library of primitive actions. Policies can become rules or scripts to be interpreted at run time, compiled into code, or implemented by instantiating data structures consulted by the application.

For more detail about the scope of research and standardisation and the differences in approach the reader is referred to the appendix.

6. The future of policy-based management

Thus far the paper has given an overview of the motivations and concepts behind policy-based management and described different approaches to policy-based management. Although the motivations seem plausible and familiar and the concepts useful, they are too abstract. In fact, to a great extent the motivations are held in common with most of the information technology and communications industry.

There is much discussion of abstract concepts, but somewhat less practical experience. The desire to provide an abstraction and incorporate flexibility, and the various issues that raises, are not unique to policy-based management. Issues from elsewhere seem to fall easily under the banner of policy-based management and much existing technology is relevant. So what is policy-based management offering that is substantially new? One could argue that policy-based management is offering a good general picture of how things might be. However, there are some serious technical and commercial issues to address before policy-based management could be considered generally feasible.

The following sections aim to provide some answers by considering the two main ambitions of policy-based management.

6.1 Policies to provide more flexible management

Policy-based management research is providing some architectural understanding and encouraging engineering of management functions in such a way as to separate hard-coded behaviour from that which needs to change more frequently. It is not necessarily providing any new technology but rather adopting existing technologies such as the rule-based paradigm. Its main contribution to achieving flexibility is in specifying the form of policy and defining functions in a policy-setting layer that helps in driving the flexible applications. The flexibility of the approach is likely to thrive only in niche areas where there is a definite requirement and no single enforcement technology is likely. This is for the following reasons.

- The consistency problem

  There must be a sufficient level of confidence that when policy-driven flexibility is exercised the result will yield correct behaviour across the distributed system, e.g. it will not generate excessive conflicts, it will offer optimal performance, it will terminate properly, behave deterministically, and exhibit confluence. Testing the behaviour of software is already difficult and it is often impossible to test every possible path through a complex piece of software. This is especially the case for the rule-based approaches advocated for policy-based management where there are more potential paths to test. Automated support may allow policies to be checked prior to enforcement, but this requires a way of formally describing behaviour and suitable algorithms and/or time to pore over this, looking for inconsistency or incompleteness. The position has not yet been reached where representation or algorithms of general applicability are available to formally check correctness. The best that can currently be achieved is to store and apply heuristic knowledge about a domain to validate policies and to provide mechanisms by which conflict can be detected and escalated to the operator for resolution.

- Commercial justification for flexibility

  Many of the approaches advocate policies as a high-level programming language for management. They give their policies all the power of a rule-based language and intend them to be of general use. Such flexibility and power is likely to be expensive to develop and potentially risk-laden for the reasons detailed above. Rather than such power being the general case, such adaptability should be limited to certain areas where there is a real requirement for this sort of flexible behaviour, and typically this will be only where the behaviour can be reasonably bounded. For example, a fraud detection system may require different fraud detection rules to be defined to keep up with the latest forms of fraud but other aspects of the functionality have less need to be reconfigured.

- Technology choice

  It is unlikely that we will achieve the flexibility of the rule-based policy approaches everywhere. More likely is that we may use policy to influence management applications than incorporate a range of technologies. There may be in-house or multiple-vendor products depending on cost, required functionality, or competitive advantage. There may be a mixture of older or newer technologies with legacy systems persisting longer than desired. Finally, different tasks in communications management are suitable for different technologies, e.g. rule-based techniques may be used for event correlation, optimisation techniques used for network design or workforce scheduling, and, on the whole, standard software tools because of the available skills.

6.2 Policies as a bridge from business goals to action

Policy-based management has made the claim that it helps address the problem of increasing scale and complexity of the systems to be managed. High-level directives which influence the behaviour of a system can be made and transformed into a consistent set of detailed changes to the system. To this end, policy-based management has provided representations of policy, storage, and tools to navigate policy information. It has addressed policy dissemination where existing management protocols have shortcomings, but currently offers few tools to automate refinement. For the following reasons early implementations will have policies closer to specific networks and systems than to high-level business-oriented policies.

- The refinement problem

  Refinement of high-level policies is likely to remain a manual process. It is a difficult process that is analogous with software design and development. It requires a large amount of human knowledge, e.g. problem domain knowledge (the managed objects, their relationships to other objects, their attributes and functions) and also solution design knowledge (achieving run-time performance, protocols for resolving conflict, etc).

  While some of the refinement can be automated by a policy support system, such as domain resolution, others, such as refining a goal into the actions necessary to achieve it, are still generally beyond automation.

- The ontology problem

  Specifying policy and achieving levels of abstraction require an ontology of the management domain, e.g. so that vendor-specific details can be hidden through choice of a common representation of the properties of different vendor offerings. There is a commercial obstacle and a technical obstacle to meeting this requirement. From a technical point of view achieving such a common representation and achieving one that covers the whole remit of management is a difficult task. This is currently happening only in certain areas and it is unlikely that we will have an ontology for all management domains in the short term. From a commercial point of view, such ontologies are created by standards bodies and industry forums which have the difficult task of trying to obtain the consensus of many different players.

7. Conclusions

This paper has identified concepts associated with policies and policy-based management and detailed the ambitions of the approach. A broad overview of some of the research and standards activities has also been provided, together with an indication of a feasible and likely outcome of these activities.

In the foreseeable future, policy-based management is not going to achieve:

- a generic policy representation,
- automated refinement of policy into action,
- formal analysis of policies to reveal conflicts,
- widely providing the level of flexibility of the rule-based approach,
- policy-basis to all areas of management and across areas.

What is more likely though is:

- a flexible approach to building management systems,
- highly domain-specific policy representations,
- generic services to help in the presentation, storage, specification, some refinement, some conflict detection, and dissemination of policies,
- domain-specific refinement and conflict detection,
- application in niche domains where a great amount of flexibility is required to enable some complex distributed system to function.

Policy-based management represents an ambitious undertaking and includes some very difficult existing issues in computer science. As such there is still much research required and applications of policy-based management in the short term are likely to be in niche domains and limited in power. Potential purchasers should be aware of this before being too drawn by some of the marketing claims for policy-based management products.

Appendix

Survey of main activities

If the reader is interested in further detail, the following sections provide more information about some of the research and standardisation efforts.

A1 Imperial College

Early work at Imperial College identified the need for policy as a way of consistently aligning components of a distributed system such as a network. They saw the need for policy to be explicitly represented so that it can be stored, viewed, processed, etc [2]. Subsequent work has addressed many issues of specifying, refining, disseminating and enforcing policy, and the platform components needed to do this.

Marriott et al [7] details the policy specification language which allow very abstract goal-type policies, constraint on action policies, and to some extent plan-of-action policies in that they let you specify a simple sequence of actions but no other control. In terms of level of abstraction the policies allow attributes with free-text values in higher level policies.

Work on refinement of policies has mostly been limited to discussion. Moffett and Sloman [8] identify the need for policy hierarchies to facilitate automatic refinement, automatic propagation of policy change, and checking that low-level policies meet high-level policies. They have attempted to categorise different types of refinement so as to suggest automatic support. There has not been much work on providing such automation and work on refinement appears to be limited to allowing ‘parent and child’ policies, and free-text values in more abstract policies.

More work has been done in the area of policy conflict. Moffett and Sloman [9] identify that that an overlap in content of subject, target and action fields is a necessary condition for potential conflict and they proceed to categorise different sorts of conflict according to the nature of the overlap. They also discuss when policy conflict can be detected and resolved and propose various strategies for resolving the different sorts of conflicts. Lupu and Sloman [10] have produced a static conflict analysis tool for detecting policy overlaps, allowing specificity of domains to resolve conflicts, and automatic specification of meta-policies to resolve apparent conflicts.

Tools and services to support policy-based management have been built. Marriott et al [7] discuss a policy service which stores and disseminates policies. Marriott and Sloman [11] describe a rule-based obligation policy interpreter. Yialesis and Sloman [12] discuss an authorisation policy interpreter. Mansouri-Samani and Sloman [13] detail the monitoring service and event-specification language which drives it. Policy and domain editors and a domain service have been constructed. Lupu and Sloman [14] expand policy-based management into role-based management where a role describes the rights and duties of particular management positions to which particular managers might be assigned. A role has a set of policies associated with it, but these alone are not sufficient to define a manager role. It is stated that they need to be combined with a specification of interaction protocols to be used between managers and concurrency constraints.

A2 University of Munich

The work here is principally that of Wies [3] whose principal contribution is his analysis of the concepts of policy-based management. In particular he categorises policies according to many different criteria in order to understand the different treatment required and from this he proposes a policy template with a number of attributes. Actions in the policy templates of Wies [3] appear to be filled with scripts with all the power of a programming language and so are more akin to plan-type statements. He defines a policy life cycle and uses this to add more attributes to policy to allow these stages. He provides analysis of policy hierarchy and refinement and some indications of how the refinement process can be supported. He proposes an architecture where the standard MIB is extended with special managed objects that provide an abstraction of common enforcement and monitoring functionality, and represents management tasks. Policies are transformed into a monitoring and enforcement unit which use these new managed objects. This involves some recompilation and linking and some interpretation by a run-time system. He also describes an example implementation based on HP Openview.

A3 University of Aachen

Koch et al [15, 16] deal with all levels of abstraction in that they describe a three-tier policy hierarchy of requirements, goals and operational policies. The first is a natural language description of policy with hypertext links to definition of key words. These are refined into the second level which consists of a number of policy attributes with free-text values. These are then refined gradually by making their values machine interpretable. The final level of policy can be transformed into IF..THEN rules and allow a number of event- or condition-driven actions with particular post-conditions set which can match with the preconditions of other policies and thus cause a chaining effect. This expressiveness makes the policies here more like the plan of action policies discussed in section 3.2. Koch et al [16] transform policies directly into rules, and event descriptions configure intelligent monitoring objects to deliver events that trigger the rules, and thus the policies define the manager's action. They address static conflict detection by mapping policies to semantic graphs and then define a tool to traverse these to look for ill-behaved policy sets.

A4 Meyer and Popien

Meyer and Popien [17, 18] are concerned with application management and their policies concern the relationship between objects and some object behaviour which is obligated or permitted. The policies do not extend the behaviour of the objects but simply constrain which of the existing behaviours are allowed by calling methods or changing the state of objects. They define a formal policy notation which is interpreted by a policy controller within the managed object. The language is quite expressive and is closer to plan-type statements in that it allows behaviours comprised from event and precondition-dependent actions or sub-behaviours. The work is seen as an example of management by delegation whereby policies are the means of delegating management directly to the managed object rather than to a management function. The policy controller takes a rule-based approach where the rules are triggered by events and subject to conditions which result in calls to the management interface offered by the object. Authorisation policies generate an access control list which is checked when the service interface of the object is called.

A5 Other research

Guerroudji [19] provides a definition of policy as specifying the relationship between managers and managed objects and a policy object with similar attributes to Imperial College, except they have a lifetime, a reaction time, refer to objects to control and resolve conflict, and have a list of compiled rule-objects associated with them. Policies are categorised as reactive (event driven) or permanent (persistent goals). A logical architecture is described whereby applications pass goals or events to a policy black box which uses policies to establish a plan comprised of rules. The architecture of the policy black box is described.

Cheh [20] describes policy as constraints on state (goal-type statement) or state transition (constraint-type statement), which guide attainment of a business objective. Implementation of a policy requires generation of a state transition plan, i.e. an action. This work considers that all policies are effectively constraints and can be enforced by similar mechanisms. Hence the categorisations of others are seen as convenient rather than implying a different mechanism. The only categorisation considered is that policy is enforced actively, requiring action to restore policy

compliance, or is enforced passively ensuring that policy is not violated in future action. Objectives are refined into a hierarchy of policies which are then refined into implementables which, in the case of active policies, specify a state transition plan. It is suggested that conflicts can be detected by failure to solve a constraint satisfaction problem created by a set of policies provided there is a finite set of states.

Boutaba and Mehaoua [21] consider policies as intermediaries easing decision making by defining how a manager can turn goals into plans of action but representing neither of these. Plans are considered to be a predefined tree of action sequences with state and policy decision points. The latter are implemented by policy predicates which are called to return the policy decision. As such the policies here offer less range of possible behaviours than rule-based approaches. They describe an engineering approach to applying policy-driven management in which one first enumerates goals, state attributes, policy attributes and available actions. Then a plan activation process is established such that events, states and policies are checked and action taken. They apply their approach to an example from ATM QoS management.

Lewis [22] implements policies as rules interpreted by a rule-interpreter. He considers the use of standard expert system conflict resolution mechanisms as appropriate and recognises the problems of rule-based systems behaving unpredictably. He also recognises that an approach to engineering policy driven applications is important. An example of configuration management of network elements is discussed where the policy rule attaches configuration records that enforce certain ranges of values to particular domains of devices and resolves conflict by following the principle that the exception overrides the rule.

Putter et al [23] consider policies as objects influencing management by obligating or authorising management to take certain actions on management resources. Policies are seen as influencing both the structure of a system of objects and their behaviour, and it is argued that these should both be represented explicitly so that they can be policy driven. They advocate meta-objects extending the base objects' behaviour as a way to enforce policies.

Alpers and Plansky [4] describe in some detail the IDSM project where policies are objects defined in Guidelines for the Definition of Managed Objects (GDMOs) which have to be specialised to contain attributes to express particular kinds of policy. For example, they describe reporting and security policies where these are enforced by creating open system interconnection (OSI) event-forwarding discriminators and access control lists. Component functions are described that are similar to those described by Imperial College except they add a refinement service and advocate different sorts of policy enforcers depending on the type of policy. They also describe an approach to engineering policy-driven applications.

A6 DMTF

The DMTF is concerned with the management of computer systems, network systems and software packages. Major activities that relate to policies are the standardisation of a Common Information Model (CIM) to support open management and the impact on it of recent work of the Directory Enabled Networking (DEN) consortium which has now been passed over to the DMTF.

DEN proposes to use a directory service, previously for looking up people, as a means of storing and accessing a wide range of information which needs to be shared. Associated with the directory is a protocol known as the lightweight directory access protocol (LDAP) for accessing the information. DEN proposes to add network, service and user extensions to the CIM and to allow the information model to be stored in a directory and accessed using LDAP [24]. One type of information that is being modelled is that of policy because it is information that can govern the operation of many devices, users, etc. The DEN specification gives particular emphasis to importance of DEN as a means to define, store, and disseminate policies.

A7 IETF

In IETF management, the information model describes the devices but does not extend well to talking about groups of objects or relationships between objects such as network topology. Protocols for configuring devices are not powerful enough to disseminate policies. Hence the IETF are aligning with DEN and the DMTF by storing such information, including policies, in a directory and using LDAP to access it.

The IETF policy framework working group exists to define a framework for definition and administration of policies and a common vendor-independent policy language. Their initial focus though is to support QoS-enabled IP networks where policy is used to ensure different packets are treated according to required QoS. This requires that consistent policies are administered and distributed to multiple-vendor multiple-function equipment. The scope may later extend to other forms of policy, although concurrently other working groups are addressing using policies. For example, the Routeing Policy System (RPS) is concerned with specification of routeing policy to constrain routeing across a network domain, its storage in a repository, and discussion of tools to ensure consistency and generating router configurations. The resource reservation protocol (RSVP) admission policy working group has discussed policies to govern reservation of resource on the network and to this end has concentrated on the common open policy service (COPS) protocol.

The policy framework working group has produced a number of Internet drafts towards these goals. This section is based on the content of two drafts in particular [25, 26]. This is a rapidly changing area and as Internet drafts are 'work in progress', the reader is referred to http://www.ietf.org for more recent material.

Generally speaking an architecture [27] is described where management tools and applications are clients of a policy repository or directory and use some protocol to access policy, e.g. LDAP. The application consists of a policy decision point which is a repository client and a policy enforcement point which checks with the decision point whenever a decision affected by policy is to be made. The decision point takes into account more dynamic information, such as network state. These latter two communicate via a policy using different protocols, e.g. COPS or simple network management protocol (SNMP). For some of the IETF policies that deal with per-packet behaviours, it is not clear whether the policy decision point needs to be consulted for every packet or whether the policies apply some configuration to the packet control software.

A policy is formally defined as an aggregation of policy rules each of which has a set of conditions and a set of actions. The content of these map to attributes and services provided by managed objects, such as testing packet header fields, and responding, for example, by marking or discarding the packet. IETF differentiated services policies are somewhere between constraint and plan-type statements having some primitive ordering, but are more like constraint type in that one cannot build complex control with them. They may conflict and should have a priority and order in which they detect conditions and execute actions. The scope of policies is intended to be just a low-level but device/vendor independent specification. It does not extend to high-level, more business-oriented policies, such as service-level agreements.

A8 OSI

The capabilities of OSI management are far richer than those of IETF management and is intended for management of managers and networks as well as individual devices. The information model is correspondingly much more expressive and it is possible to use it to model policies and topology. The protocol is also much more powerful with better event handling and scoped action, which would tend towards more efficiency in the treatment of policies. Hence concepts of policies and domains are more easily catered for by extending existing standards. The concepts of policies and domains are introduced in the OSI system management overview [6]. Here a policy is considered to be an identifiable specification that can be evaluated with respect to managed objects and generates a violation event. The concept of domain allows enumeration or selection of a number of managed objects.

ITU X.749 [28] is concerned with the modelling of policy and domain objects and the messages and content that would need to pass across a management interface to manipulate these objects. The standard is not therefore concerned with the details or scope of the implementation at either end of that interface, e.g. how conflict is resolved. The standard defines a policy object with only administrative fields from which specialisation is necessary to define specific policy types. The standard only defines one particular kind of policy sub-class, namely value assertion policy. This uses a logical combination of value assertions which specify, for some managed object, what operations, notifications and replies are allowed and with what parameter values. Notifications can take place on violation, or violations can be trapped and prevented and the evaluation of the policy can be scheduled. These policies can describe constraints on state and constraints on action, but do not describe plans of action. The level of abstraction is at the lowest level only (i.e. the details of managed objects), cannot describe more abstract policies, and does not consider policy hierarchies or refinement. These policies are more about influencing the behaviour of the managed system and not about defining it.

A9 ODP

The Open Distributed Processing (ODP) reference model aims to provide a co-ordinating framework for standardisation, enabling distributed, heterogeneous systems to interwork. It describes an object modelling approach to system specification and discusses system specification from several viewpoints. It describes at a high level the infrastructure required to provide interworking.

ITU-T X.901 [29] introduces policies as authorisations or obligations and a domain is defined as a group of objects where some aspect of the behaviour is controlled by the same authority, e.g. security, naming. Each domain is controlled by a common domain controlling object so that policy and domain membership can be controlled through interaction with this object rather than managers having to deal with each managed object individually.

The policy and domain objects appear in discussion of the enterprise viewpoint of system specification [30] which is concerned with objects, object communities and roles of the objects expressed in terms of permissions, obligation, or prohibition policies. Here policies govern interactions between those objects, creation, use and deletion of resources used, configuration of objects, assignment of roles, and contracts with the system environment.

A10 OMG

The OMG Common Object Request Broker Architecture (CORBA) specification is concerned with the implementation of the ODP engineering viewpoint and in providing a number of transparencies for the programmer, in particular distribution transparencies. The concepts of policies and domains inherit much from the ODP work. They appear in the specification [31] as an interface to configure the object request broker (ORB) and services, [31] and this capability is extended to applications as a common facility [32], and defined in detail in an adopted request for comment [33]. Typically the specifications are concerned with defining the object interfaces to associate policies with application objects and are not concerned overly with the attributes of the policy or the internal workings of the policy-driven application.

Two types of policy are permitted — namely initialisation and validation — which govern the state of application objects at creation and the states into which they can be modified. There is no detail about the attributes of the policies, only that they must provide a certain interface that can be called and allow initialisation or validation of attributes. It is a matter for the developer (whose policy-driven application object must inherit from base classes defined in the specification) when the call is made. The facility provides means by which application objects can become associated with policy regions (domains), and policy regions with policies and with instance managers that create application objects. The policies can be written in any programming language with all the expressiveness that this entails, but must be compiled. No notion of describing and refining from abstract policies is considered, and possible policy conflict is mentioned but not addressed.

Acknowledgements

Thanks are due to Jim Hardwicke, Sagar Gordhan and Manooch Azmoodeh for help in reviewing this paper and discussion of many of the issues.

References

1 Sloman M: ‘Policy driven management for distributed systems’, Journal of Networks and Systems Management, 2, No 4, pp 333—360, Plenum Press (December 1994).

2 Moffett J D and Sloman M: ‘The representation of policies as system objects’, Conf on Organisational Computer Systems (COCS’91) in SIGOIS Bulletin, 12, Nos 2 and 3, pp 171—184 (1991).

3 Wies R: ‘Policies in integrated network and systems management — methodologies for the definition, transformation and application of management policies’, Verlag Shaker, Aachen (1995).

4 Alpers B and Plansky H: ‘Concepts and application of policy-based management’, Proceedings Integrated Network Management, IV, (May 1995).

5 Becker K and Holden D: ‘Specifying the dynamic behaviour of management systems’, Journal of Networks and Systems Management, 1, pp 281—298, Plenum Press (1993).

6 ITU-T Recommendation X.701: ‘Information technology — Open Systems Interconnection — Systems management overview’, (1997).

7 Marriott D, Sloman M and Yialelis N: ‘Management policy service for distributed systems’, Imperial College Research Report, DoC 95/10 (1995).

8 Moffett J D and Sloman M: ‘Policy hierarchies for distributed systems management’, IEEE JSAC Special issue on network management, 11, No 9 (December 1993).

9 Moffett J D and Sloman M: ‘Policy conflict analysis in distributed system management’, Journal of Organizational Computing (April 1993).

10 Lupu E and Sloman M: ‘Conflict analysis for management policies’, Proceedings Integrated Network Management, V, (May 1997).

11 Marriot D and Sloman M: ‘Implementation of a management agent for interpreting obligation policy’, Distributed Systems Operations and Management (October 1996).

12 Yialelis N and Sloman M: ‘A security framework supporting domain based access control in distributed systems’, IEEE Proceedings of the Internet Society Symposium on Network and Distributed Systems Security, San Diego (February 1996).

13 Mansouri-Samani M and Sloman M: ‘GEM a generalised event monitoring language for distributed systems’, Distributed Systems Engineering Journal 4, No 2 (June 1997).

14 Lupu E and Sloman M: ‘Towards a role-based framework for distributed systems management’, Journal of Networks and Systems Management, 5, No 1 (March 1997).

15 Koch T and Kramer B: ‘Towards a comprehensive distributed systems management’, Proceedings 3rd IFIP Internation Conference on Open Distributed Processing — ICODP ’95, pp 259—270 (February 1995).

16 Koch T, Krell C and Kramer B: ‘Policy definition language for automated management of distributed systems’, Proceedings of IEEE International Workshop on Systems Management, pp 55—64 (June 1996).

17 Meyer B, Anstoz F and Popien C: ‘Towards implementing a policy based systems management’, Distributed Systems Engineering, 3, No 2, pp 78—85 (June 1996).

18 Meyer B and Popien C: ‘Flexible management of ANSAware applications’, Proceedings 3rd IFIP International Conference on Open Distributed Processing, pp 271—282 (February 1995).

19 Guerroudji N: ‘A management policies framework’, Proc of IEEE Int Workshop on System Management, IEEE Comput Soc Press, pp 40—46 (June 1996).

20 Cheh G: ‘A generic approach to policy description in system management’, HP Labs Technical Report HPL-97-82 (1997).

21 Boutaba R and Mehaoua A: ‘Applying the policy concept to the management of ATM networks’, Proc of IEEE Int Workshop on System Management, IEEE Comput Soc Press, pp 47—54 (June 1996).

22 Lewis L: ‘Implementing policy in enterprise networks’, IEEE Communications magazine, pp 50—55 (January 1996).

23 Putter P, Bishop J and Roos J: ‘Towards policy driven systems management’, Proceedings Integrated Network Management, IV, (May 1995).

24 Judd S, and Strassner J: ‘Directory-enabled networks’, version 3.0c5 (1998) — http://murchiso.com/den/specifications/directory-enabled-networks-v3c5-lastcall-no-figs.pdf

25 Strassner J and Ellesson E: ‘Internet draft: Terminology for describing network policy and services’, (August 1998) — http://www.ietf.org/internet-drafts/draft-rajan-policy-qosschema-00.txt

26 Rajan R, Martin J C, Kamat S, See M, Chaudhury R, Verma D, Powers G and Yavatkar R: 'Internet draft: Schema for differentiated services and integrated services in networks', (October 1998) — http://www.ietf.org/internet-drafts/draft-strassner-policy-terms-00.txt

27 Masullo M and Calo S: 'Policy management: an architecture and approach', IEEE Workshop on Systems Management (1993).

28 ITU-T Recommendation X.749: 'Information technology — Open Systems Interconnection — Systems management: management domain and management policy, management function', (1997).

29 ITU-T Recommendation X.901: 'Information technology — Open Distributed Processing — Reference Model: Overview', (August 1997).

30 ITU-T Recommendation X.902: 'Information technology — Open Distributed Processing — Reference Model: Architecture', (November 1995).

31 OMG: 'The Common Object Request Broker: Architecture and Specification 2.2', (1998).

32 OMG: 'CORBA facilities Architecture Specification', (1995).

33 'X/Open OMG Request for Comment Submission, Systems Management: Common Management Facilities, Volume 1, Version 2', (December 1995).

Michael Cox graduated from Brunel University in 1987 with a BSc in Physics and Computer Science. He joined BT Laboratories working in the transmission domain surveillance (TDS) programme. He was primarily responsible for the design and implementation of a prototype PDH fault correlation expert system through several live field trials and then integrating it into the TDS fault manager product. In 1995 he moved to the network management research team where he exploited ideas from agent and constraint satisfaction technologies for the performance management of ATM networks. In 1997 he obtained an MSc in Computer Science (Artificial Intelligence) at the University of Essex. More recently he has been involved in research into policy-based management and management of layered networks.

Rob Davison graduated from Bristol University with a degree in Computer Systems Engineering in 1988. He joined BT's speech and language division and spent two years researching the application of AI techniques to spoken dialogue systems, before transferring into the network management research area. Since then he has worked on a range of research and development projects covering areas such as fault diagnosis, service provisioning, broadband performance management and distributed computing. He currently leads the network management research group with interests in new architectures for network management, open network control and the management of future ATM and IP networks.